AI 协作反思日志 — Phase 2 体系结构设计

团队：Camera项目组 日期：2026-05-04

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
架构基础概念理解	Gemini, ChatGPT, Claude	询问“什么是架构”“MVC 和分层架构有什么区别”，要求用容易理解的方式解释课堂中的架构风格	5	30%
架构辩论赛 Prompt 设计	Gemini, ChatGPT, Claude	以成员 C 的身份，提供约拍服务平台项目介绍，要求 AI 对比纯 MVC、纯分层架构、微服务架构、MVC + 分层架构四种方案	5	85%
候选架构对比与收敛	ChatGPT, Claude	结合课程项目、小团队、10 周周期、APP 方向和约拍业务闭环，要求 AI 判断哪种架构更适合本项目	4	35%
任务二模块划分辅助	ChatGPT, Claude	提供上一阶段需求规格说明书和用户故事，要求 AI 按任务二标准重新设计模块划分、模块依赖、接口和技术选型	5	30%

2. AI 助力分析（Q1）

本阶段 AI 最大的贡献是：帮助团队把“架构”这个抽象概念转化成可以理解和执行的任务。最开始团队成员对 MVC、分层架构、微服务、事件驱动等概念关系仍不熟悉，也不完全清楚该如何把架构设计落到实处。AI 通过“房屋结构”“系统怎么拆、每块负责什么、它们怎么协作”等类比，帮助我们理解架构并不是某一行代码或某个页面设计，而是系统整体组织方式。

在架构风格选择上，AI 帮助我们明确：本项目虽然包含用户、服务方、作品、需求、档期、沟通、订单、交付、评价、审核等多个业务模块，但这并不意味着必须采用微服务架构。对于 10 周左右的课程项目和第一次接触架构设计的小团队来说，微服务会引入服务拆分、服务通信、部署运维、数据一致性等额外复杂度，反而可能影响核心功能交付。因此，AI 给出的主要建议是采用 MVC + 分层架构，在一个后端应用中按业务模块组织代码。

不同 Prompt 策略下 AI 回答质量的差异：一开始只是泛泛询问“什么架构合适”时，AI 的回答更偏概念解释和通用建议；当我们补充约拍平台的具体背景后，AI 能够围绕“找人—沟通—预约—交易—交付—评价”的业务闭环进行分析；当我们进一步明确“要做 APP、不是微信小程序”“任务二以模块划分和技术选型为准”“不要过度迎合用户、以正确判断为第一标准”之后，AI 的回答明显更贴近作业要求，减少了无关细节，重点转向模块职责边界、依赖关系、接口方式和技术选型。

3. AI 误导分析（Q2）

AI 在架构建议中的典型问题：

- 问题1：AI 容易把“模块划分”说得像“微服务拆分”，导致初学者误以为用户模块、订单模块、评价模块都应该是独立部署的服务。
  如何发现：在讨论任务二时，我们发现项目虽然有很多业务模块，但团队周期短、经验不足，且作业要求只是模块划分、接口定义和技术选型，并没有要求独立部署多个服务。如果真的按微服务做，会明显超出课程项目能力范围。
  如何修正：后续 Prompt 中明确要求 AI 区分“业务模块”和“微服务”，最终将这些模块定义为同一个后端项目中的功能模块，而不是多个独立服务。
- 问题2：AI 容易推荐过多中间件或高级技术，例如消息队列、服务网关、完整支付系统、复杂推荐算法等。
  如何发现：结合任务二要求，技术选型只需要说明前端、后端、数据库、中间件和部署方式，不需要把所有可能用到的技术都堆进去。项目核心目标是完成可运行 MVP，而不是搭建大型商业平台。
  如何修正：我们将消息队列、复杂推荐算法、真实担保支付、提现结算等内容降级为“后续扩展”或“预留接口”，MVP 阶段只保留 REST API、MySQL、文件存储、可选 Redis、WebSocket 或轮询等更可落地的方案。
- 问题3：AI 容易延续上一阶段文档中的“微信小程序优先”假设，而没有自动意识到我们当前已经确定做 APP。
  如何发现：我们在任务二讨论中明确指出“我们要做 APP，不是什么老是弄微信小程序”。这说明旧需求文档中的终端假设已经不完全适用。
  如何修正：在后续架构方案中，将前端终端调整为移动端 APP，并把技术选型改为 Flutter / React Native 一类跨平台 APP 方案，同时保留后台管理 Web，而不是继续默认微信小程序。
- 问题4：AI 容易用通用电商平台思路套约拍平台。
  如何发现：上一阶段 AI 盲区分析已经发现，AI 容易遗漏约拍行业特有问题，例如样片与客片差异、原片与精修区分、双向评价、肖像权、成片质量争议、线下见面安全等。
  如何修正：在任务二模块划分时，我们没有只按通用电商的“用户、商品、订单、评价”拆分，而是保留了作品集模块、档期模块、消息与报价模块、作品交付模块、评价与信用模块、后台审核与仲裁模块，使架构更贴合约拍业务。

4. Prompt 改进计划（Q3）

上阶段（P1）提出的改进措施，验证结果：上阶段我们已经发现，AI 在没有真实调研数据时，容易生成格式完整但不一定真实的需求，例如编造具体佣金比例、并发量、退款比例，或者遗漏约拍行业特有的信任与安全问题。因此本阶段我们在使用 AI 时，主动提供了上一阶段需求规格说明书、用户故事和 AI 需求盲区分析表，让 AI 以这些材料为基础进行架构设计。验证结果是：AI 输出的模块划分比单纯提问时更贴合实际业务，能够围绕作品集、档期、报价、订单、交付、评价、仲裁等核心流程展开，而不是只给出空泛的架构术语。

本阶段新的改进措施：后续在使用 AI 辅助详细设计或代码生成时，Prompt 必须同时包含四类信息：第一，项目背景和核心业务闭环；第二，团队人数、开发周期、技术能力等现实约束；

第三，当前任务的边界，例如任务二只需要模块划分和技术选型，不需要完整数据库设计；第四，明确要求 AI 区分“必须实现”“可以后续扩展”“暂不考虑”的内容。这样可以减少 AI 的过度设计，避免把大型商业系统方案套到课程项目中。

此外，后续使用 AI 时应要求 AI 输出“选择理由 + 不选择理由 + 风险提醒”。例如技术选型时，不仅要问“用什么技术”，还要问“为什么不用更复杂的技术”“这个选择会带来什么风险”。这种提问方式比直接让 AI 给方案更有助于团队理解架构权衡。

5. 本阶段的核心工程判断

决策内容：本项目最终倾向采用 MVC + 分层架构，不采用微服务架构。系统整体按照移动端 APP、后台管理端、后端服务、数据库和文件存储进行组织；后端采用 Controller、Service、Repository/Mapper 的分层方式；业务上划分为用户与身份、服务方主页、作品集、需求发布、搜索与匹配、档期、消息与报价、订单与交易、作品交付、评价与信用、后台审核与仲裁等模块。各模块在同一个后端应用中实现，通过 Service 层进行协作。

为什么 AI 无法做这个决策：AI 可以解释 MVC、分层架构、微服务、事件驱动等方案的优缺点，也可以根据项目描述生成模块划分建议，但它无法真正替代团队做最终工程判断。因为最终选择不仅取决于理论上的架构优劣，还取决于团队成员是否真的能理解和实现、10 周内能否完成、APP 端和后端是否能顺利联调、是否有足够时间处理订单状态、图片上传、聊天报价、交付评价、后台审核等核心流程。

本阶段我们真正需要判断的是“能不能做出来”，而不是“架构听起来是否高级”。微服务架构在扩展性上更强，但对当前课程项目来说开发和部署成本过高；事件驱动架构适合复杂异步流程，但完整引入消息队列会增加学习负担；纯 MVC 或纯分层架构又无法充分表达 APP、后台和后端业务模块之间的关系。因此团队最终采用 MVC + 分层架构，并在业务模块内部保持清晰职责边界。这一判断必须结合团队真实能力和项目交付压力，不能完全由 AI 决定。